l

INSTITUTO TECNOLÓGICO DE MÉXICO

CAMPUS ORIZABA

“ACTIVIDAD 5”

EQUIPO 9

GARCIA CEDEÑO IRVING EMANUEL

ROJAS BONILLA SANTIAGO

MARTINEZ ROSAS JOSE ANGEL

MATERIA: PROGRAMACION ORIENTADA A OBJETOS

GRUPO: VESPERTINO 1-2 PM

MAESTRO: RAFAEL HERRERA GARCIA

FECHA DE ENTREGA: 16 DE ABRIL DE 2026

Objetivo: implementar los métodos equals, hashCode, compareTo en los problemas vistos de la actividad 2.

Problema 1:

En cierta empresa de fabricación de ropa, se requiere sistematizar el proceso, se fabrican diversidad de prendas de vestir tanto para caballeros como damas, no se fabrica ropa de niños, de cada prenda se tiene un registro de modelo, tela, costo de producción por pieza, genero (masculino, femenino o mixto), temporada (primavera, verano, otoño o invierno), cuando se produce un lote de cada prenda, se registra número de lote, número de piezas, fecha de fabricación. De cada lote debe calcularse el costo de producción del lote, así como el monto de recuperación que se obtendrá después de su venta, el precio de venta por pieza es siempre 15% más del costo de producción y por lote completo es por cada pieza el 5% más del costo de producción.

Análisis Del Problema:

En este problema vemos que es sobre una fábrica de ropa, lo cual lo dividimos en dos clases:

Clase Prenda:

La clase Prenda representa cada tipo de prenda que fabrica la empresa. En esta clase se guardan datos como el modelo, la tela utilizada, el costo de producción por pieza, el género y la temporada. Una prenda puede tener uno o varios lotes de producción, ya que cada vez que se fabrica se registra un nuevo lote.

Además, la clase permite calcular el precio de venta por pieza, el cual se obtiene aumentando el 15% al costo de producción.

Clase Lote:

La clase Lote representa cada lote de producción de una prenda. Cada lote tiene su número de lote, el número de piezas producidas y la fecha de fabricación.

A partir de esta información se puede calcular el costo total de producción del lote y el monto de recuperación después de la venta.

Clase prenda:

Prenda			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
modelo	String	1	si
tela	String	1	si
costoProduccionPZ	Double	1	si
genero	String	1	si
temporada	String	1	si

Operaciones:

Prenda			
operación	Tipo de dato(retorno)	argumentos	Alcance
constructor		(Modelo, tela, genero, temporada)	Public
calcularPrecioVenta	double		public

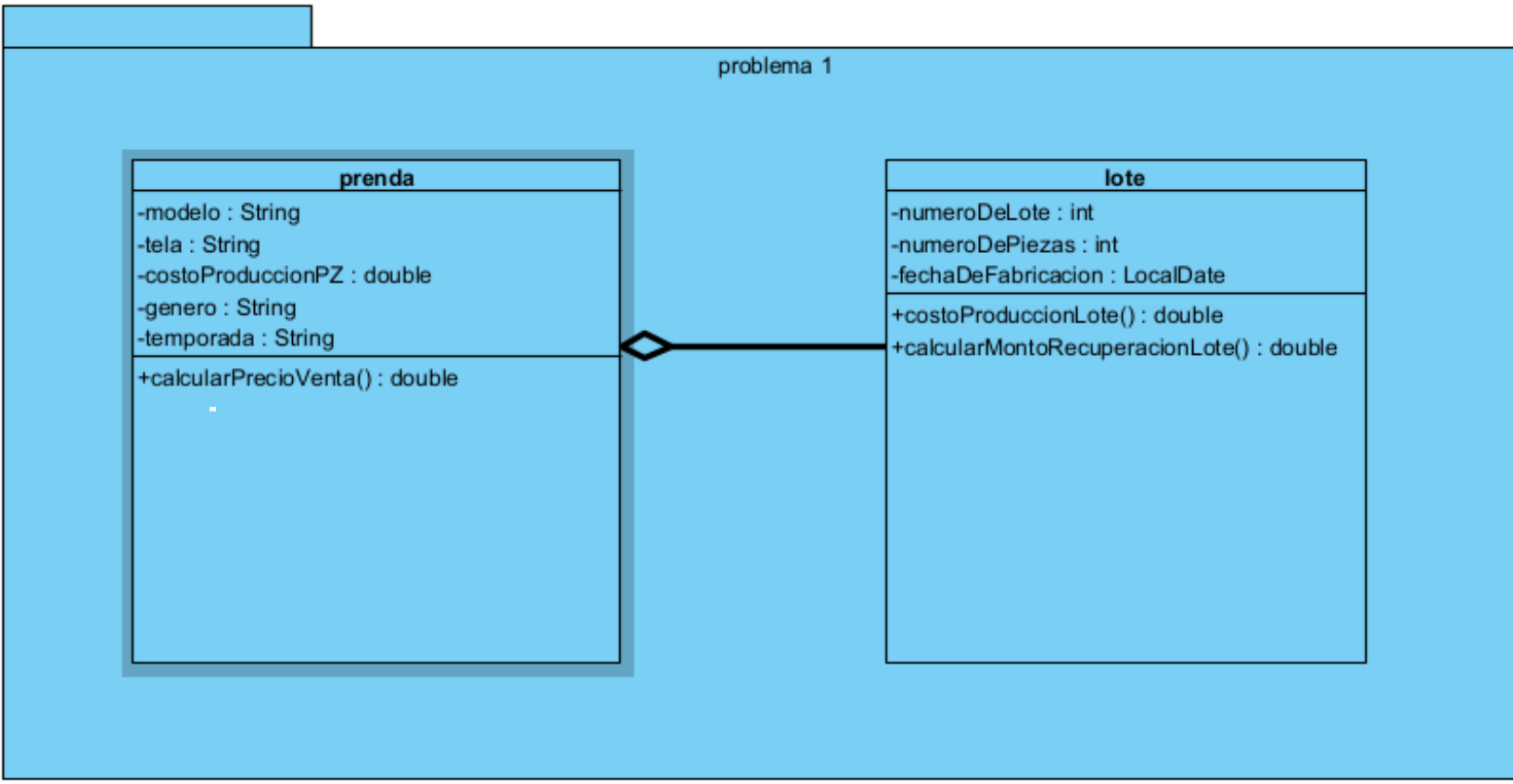
Clase lote:

Lote			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
numeroDeLote	Int	1	Si
númeroDePiezas	int	1	Si
fechaDeFabricación	LocalDate	1	Si

Operaciones:

Lote			
operación	Tipo de dato(retorno)	argumentos	Alcance
constructor		(numeroDeLote, numeroDePiezas, fechaDeFabricacion)	
costoProduccionLote	double		public
calcularMontoRecuperacionLote	double		public

Diagrama:


problema 1
prenda
-modelo : String
-tela : String
-costoProduccionPZ : double
-genero : String
-temporada : String
+calcularPrecioVenta() : double
lote
-numeroDeLote : int
-numeroDePiezas : int
-fechaDeFabricacion : LocalDate
+costoProduccionLote() : double
+calcularMontoRecuperacionLote() : double

Código clase prenda:

```java
package Fabrica.Ropa;

import java.util.Objects;

public class Prenda implements Comparable<Prenda> {  5 usages  Irving Emanuel Garcia Cedeño
    private String Modelo;  7 usages
    private String Tela;  4 usages
    private double PrecioVentaPZ;  4 usages
    private double PrecioVentaLote;  2 usages
    private String Genero;  4 usages
    private String Temporada;  4 usages

    public Prenda(String modelo, String tela, double precioVentaPZ, double precioVentaLote, String genero, String temporada) {
        this.Modelo = modelo;
        this.Tela = tela;
        this.PrecioVentaPZ = precioVentaPZ;
        this.PrecioVentaLote = precioVentaLote;
        this.Genero = genero;
        this.Temporada = temporada;
    }

    public String getModelo() {  no usages  Irving Emanuel Garcia Cedeño
        return Modelo;
    }
```

```java
public class Prenda implements Comparable<Prenda> {
    public void setModelo(String modelo) {  no usages
        Modelo = modelo;
    }

    public String getTela() {  no usages  Irving Emanuel
        return Tela;
    }

    public void setTela(String tela) {  no usages  Irvi
        Tela = tela;
    }

    public String getGenero() {  no usages  Irving Emanu
        return Genero;
    }

    public void setGenero(String genero) {  no usages
        Genero = genero;
    }

    public String getTemporada() {  no usages  Irving Em
        return Temporada;
    }
```

Lote.java Prenda.java

```java
public class Prenda implements Comparable<Prenda> {  5 usages  Irving Emanuel
    public void setTemporada(String temporada) {  no usages  Irving Emanuel Ga
        Temporada = temporada;
    }

    public double precioVentaPZ() {  2 usages  Irving Emanuel Garcia Cedeño
        return PrecioVentaPZ * 1.15;
    }

    public double precioventaLote() {  no usages  Irving Emanuel Garcia Cedeño
        return PrecioVentaPZ * 1.05;
    }
    @Override  Irving Emanuel Garcia Cedeño
    public int compareTo(Prenda p) {
        return Double.compare(this.precioVentaPZ(), p.precioVentaPZ());
    }
    @Override  Irving Emanuel Garcia Cedeño
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Prenda)) return false;
        Prenda p = (Prenda) obj;
        return Modelo.equals(p.Modelo);
    }
    @Override  Irving Emanuel Garcia Cedeño
    public int hashCode() {
        return Objects.hash(Modelo);
    }
```

Lote.java Prenda.java

```java
public class Prenda implements Comparable<Prenda> {  5 usage

    @Override  Irving Emanuel Garcia Cedeño
    public String toString() {
        return "prenda{" +
                "modelo='" + Modelo + '\'' +
                ", Tela='" + Tela + '\'' +
                ", precioventaPZ=" + PrecioVentaPZ +
                ", precioVentaLote=" + PrecioVentaLote +
                ", genero='" + Genero + '\'' +
                ", temporada='" + Temporada + '\'' +
                '}';
    }
```

Código clase lote:

```java
package Fabrica.Ropa;

import java.time.LocalDate;

public class Lote {  no usages  Irving Emanuel Garcia Cedeño
    private int NumDeLote;  4 usages
    private int NumDePIezas;  6 usages
    private LocalDate FechadeFabricacion;  4 usages
    private double CostoProduccionPz;  5 usages

    public Lote(int numDeLote, int numDePIezas, LocalDate fechadeFabricacion, double costoProduccionPz) {  no usage
        this.NumDeLote = numDeLote;
        this.NumDePIezas = numDePIezas;
        this.FechadeFabricacion = fechadeFabricacion;
        this.CostoProduccionPz = costoProduccionPz;
    }

    public int getNumDeLote() {  no usages  Irving Emanuel Garcia Cedeño
        return NumDeLote;
    }

    public void setNumDeLote(int numDeLote) {  no usages  Irving Emanuel Garcia Cedeño
        NumDeLote = numDeLote;
    }

    public int getNumDePIezas() {  no usages  Irving Emanuel Garcia Cedeño
        return NumDePIezas;
```

```java
package Fabrica.Ropa;

import java.time.LocalDate;

public class Lote {  no usages  Irving Emanuel Garcia Cedeño
    private int NumDeLote;  4 usages
    private int NumDePIezas;  6 usages
    private LocalDate FechadeFabricacion;  4 usages
    private double CostoProduccionPz;  5 usages

    public Lote(int numDeLote, int numDePIezas, LocalDate fechadeFabricacion, double costoProduccionPz) {
        this.NumDeLote = numDeLote;
        this.NumDePIezas = numDePIezas;
        this.FechadeFabricacion = fechadeFabricacion;
        this.CostoProduccionPz = costoProduccionPz;
    }

    public int getNumDeLote() {  no usages  Irving Emanuel Garcia Cedeño
        return NumDeLote;
    }

    public void setNumDeLote(int numDeLote) {  no usages  Irving Emanuel Garcia Cedeño
        NumDeLote = numDeLote;
    }
```

Lote.java × Prenda.java

```java
public class Lote {  no usages  Irving Emanuel Garcia Cedeño
    public void setNumDeLote(int numDeLote) {  no usages  Irving Emanuel Ga
        NumDeLote = numDeLote;
    }

    public int getNumDePIezas() {  no usages  Irving Emanuel Garcia Cedeño
        return NumDePIezas;
    }

    public void setNumDePIezas(int numDePIezas) {  no usages  Irving Emanu
        NumDePIezas = numDePIezas;
    }

    public LocalDate getFechadeFabricacion() {  no usages  Irving Emanuel G
        return FechadeFabricacion;
    }

    public void setFechadeFabricacion(LocalDate fechadeFabricacion) {
        FechadeFabricacion = fechadeFabricacion;
    }

    public double getCostoProduccionPz() {  no usages  Irving Emanuel Garcia
        return CostoProduccionPz;
    }

    public void setCostoProduccionPz(double costoProduccionPz) {  no usa
        CostoProduccionPz = costoProduccionPz;
    }
```

Lote.java × Prenda.java

```java
public class Lote {  no usages  Irving Emanuel Garcia Cedeño
    public void setCostoProduccionPz(double costoProduccionPz) {
        CostoProduccionPz = costoProduccionPz;
    }

    public double costoProduccionLote(){  no usages  Irving Emanuel Garc
        return NumDePIezas*CostoProduccionPz;
    }
    public double calcularMontoRecuperacion(){  no usages  Irving Emar
        double precioVenta=CostoProduccionPz*1.15;
        return NumDePIezas*precioVenta;
    }

    @Override  Irving Emanuel Garcia Cedeño
    public String toString() {
        return "lote{" +
                "numDeLote=" + NumDeLote +
                ", numDePIezas=" + NumDePIezas +
                ", fechadeFabricacion=" + FechadeFabricacion +
                '}';
    }
}
```

Prueba:

```
jshell> Prenda p1 = new Prenda("Camisa", "Algodon", 100, 1500, "Hombre", "Verano");
p1 ==> prenda{modelo='Camisa', Tela='Algodon', precioven ... mbre', temporada='Verano'}

jshell> Prenda p2 = new Prenda("Pantalon", "Mezclilla", 200, 2500, "Hombre", "Invierno");
p2 ==> prenda{modelo='Pantalon', Tela='Mezclilla', preci ... re', temporada='Invierno'}

jshell> Prenda p3 = new Prenda("Camisa", "Algodon", 100, 1500, "Hombre", "Verano");
p3 ==> prenda{modelo='Camisa', Tela='Algodon', precioven ... mbre', temporada='Verano'}

jshell>

jshell> p1.equals(p3);
$57 ==> true

jshell> p1.equals(p2);
$58 ==> false

jshell> p1.compareTo(p2);
$59 ==> -1

jshell> p2.compareTo(p1);
$60 ==> 1

jshell> p1.precioVentaPZ();
$61 ==> 114.99999999999999

jshell> p1.precioventaLote();
$62 ==> 105.0

jshell> ArrayList<Prenda> prendas = new ArrayList<>();
prendas ==> []
```

```
jshell> prendas.add(p1);
$64 ==> true

jshell> prendas.add(p2);
$65 ==> true

jshell> prendas.add(p3);
$66 ==> true

jshell>

jshell> Collections.sort(prendas);

jshell> prendas;
prendas ==> [prenda{modelo='Camisa', Tela='Algodon', precioventaPZ=100.0, precioVentaLote=1500.0, genero='Hombre', temporada='Verano'}, prenda{modelo='Camisa'
, Tela='Algodon', precioventaPZ=100.0, precioVentaLote=1500.0, genero='Hombre', temporada='Verano'}, prenda{modelo='Pantalon', Tela='Mezclilla', precioventaPZ
=200.0, precioVentaLote=2500.0, genero='Hombre', temporada='Invierno'}]

jshell> Lote l1 = new Lote(1, 10, LocalDate.now(), 50);
l1 ==> lote{numDeLote=1, numDePIezas=10, fechadeFabricacion=2026-04-14}

jshell> Lote l2 = new Lote(2, 20, LocalDate.now(), 60);
l2 ==> lote{numDeLote=2, numDePIezas=20, fechadeFabricacion=2026-04-14}

jshell> l1.costoProduccionLote();
$71 ==> 500.0

jshell> l1.calcularMontoRecuperacion();
$72 ==> 574.9999999999999
```

```
jshell> l1;
l1 ==> lote{numDeLote=1, numDePIezas=10, fechadeFabricacion=2026-04-14}

jshell> l2;
l2 ==> lote{numDeLote=2, numDePIezas=20, fechadeFabricacion=2026-04-14}
```

Análisis del código:

En esta parte del programa se creó un objeto llamado miPrenda, al cual se le asignaron los valores correspondientes a los atributos definidos en la clase Prenda, como el modelo, tipo de tela, costo de producción por pieza, género y temporada. Posteriormente se imprimen los datos del objeto para verificar que la información se haya guardado correctamente.

También se utiliza el método calcularPrecioVenta(), el cual permite obtener el precio de venta por pieza aumentando el 15% al costo de producción, tal como lo indica el problema.

Comparamos utilizando el método equals(). Cuando los atributos de las prendas son diferentes, el resultado es false, lo que indica que no son iguales. Esto demuestra que el método está correctamente implementado comparando los atributos, de igual forma, se incluye hashCode().

También se utiliza el método compareTo(), el cual permite comparar dos prendas, el resultado es -1, significa que una prenda es menor (el costo de producción). Esto permite ordenar objetos.

(Lote):

En esta parte se creó un objeto llamado miLote, al cual se le asignaron los valores definidos en la clase Lote, como el número de lote, el número de piezas y la fecha de fabricación.

Después se utilizan los métodos de la clase para realizar los cálculos solicitados en el problema. El método costoProduccionLote() calcula el costo total de producción multiplicando el número de piezas por el costo de producción por pieza.

Por otro lado, el método calcularMontoRecuperacionLote() calcula el monto de recuperación del lote considerando el precio de venta por pieza, el cual incluye un aumento sobre el costo de producción.

Finalmente, se imprimen los resultados para comprobar que los cálculos se realizaron correctamente, así como el correcto funcionamiento de los métodos de comparación y ordenamiento implementados.

Problema 2:

Una empresa agrícola ha decidido sistematizar el control de la producción frutal que tiene, para lo cual de cada fruta se describe su nombre, la extensión del terreno (hectáreas) dedicadas a su cultivo y producción, el tiempo de cosecha (Cada fruta puede tener 1 o más periodos de cosecha), la cantidad estimada a cosechar por hectárea en cada periodo (cada periodo por condiciones de estación climática puede tener diferentes cantidades estimadas), costo de producción promedio por tonelada y el precio de venta promedio por tonelada. Por cada fruta se puede agregar o eliminar un periodo de cosecha, se puede obtener el costo total de un periodo, así como las ganancias estimadas en un periodo.

Análisis Del Problema:

Ahora para este problema fue una empresa agrícola que controla la producción de frutas y ocupamos dos clases que son:

Clase Fruta: La clase Fruta representa cada tipo de fruta que produce la empresa agrícola. En esta clase se almacenan datos como el nombre de la fruta, la cantidad de hectáreas destinadas a su cultivo, el costo de producción por tonelada y el precio de venta por tonelada.

Además, cada fruta puede tener uno o varios periodos de cosecha, por lo que se utiliza una lista para almacenar los diferentes periodos registrados.

Las operaciones de esta clase permiten agregar o eliminar periodos de cosecha para cada fruta.

Clase Periodo Cosecha: La clase PeriodoCosecha representa cada periodo en el que se realiza la cosecha de una fruta. Esta clase contiene el nombre del periodo y la cantidad estimada de toneladas que se pueden producir por hectárea.

También incluye operaciones que permiten calcular la producción total, el costo de producción y las ganancias estimadas durante ese periodo de cosecha.

Clases Fruta

fruta			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
nombreFruta	String	1	Si
hectáreas	Double	1	Si
costoProducciónToneladas	Double	1	Si
precioVentaToneladas	Double	1	Si
periodos	ArrayList<periodoCosecha>	0	si

Operaciones:

Fruta			
Operación	Tipo de dato (retorno)	Argumentos	Alcance
constructor		nombreFruta, hectáreas, costoProduccionToneladas, periodos	
agregarPeriodo	void	periodoCosecha, periodo	public
eliminarPeriodo	void	Periodo, periodoCosecha	public

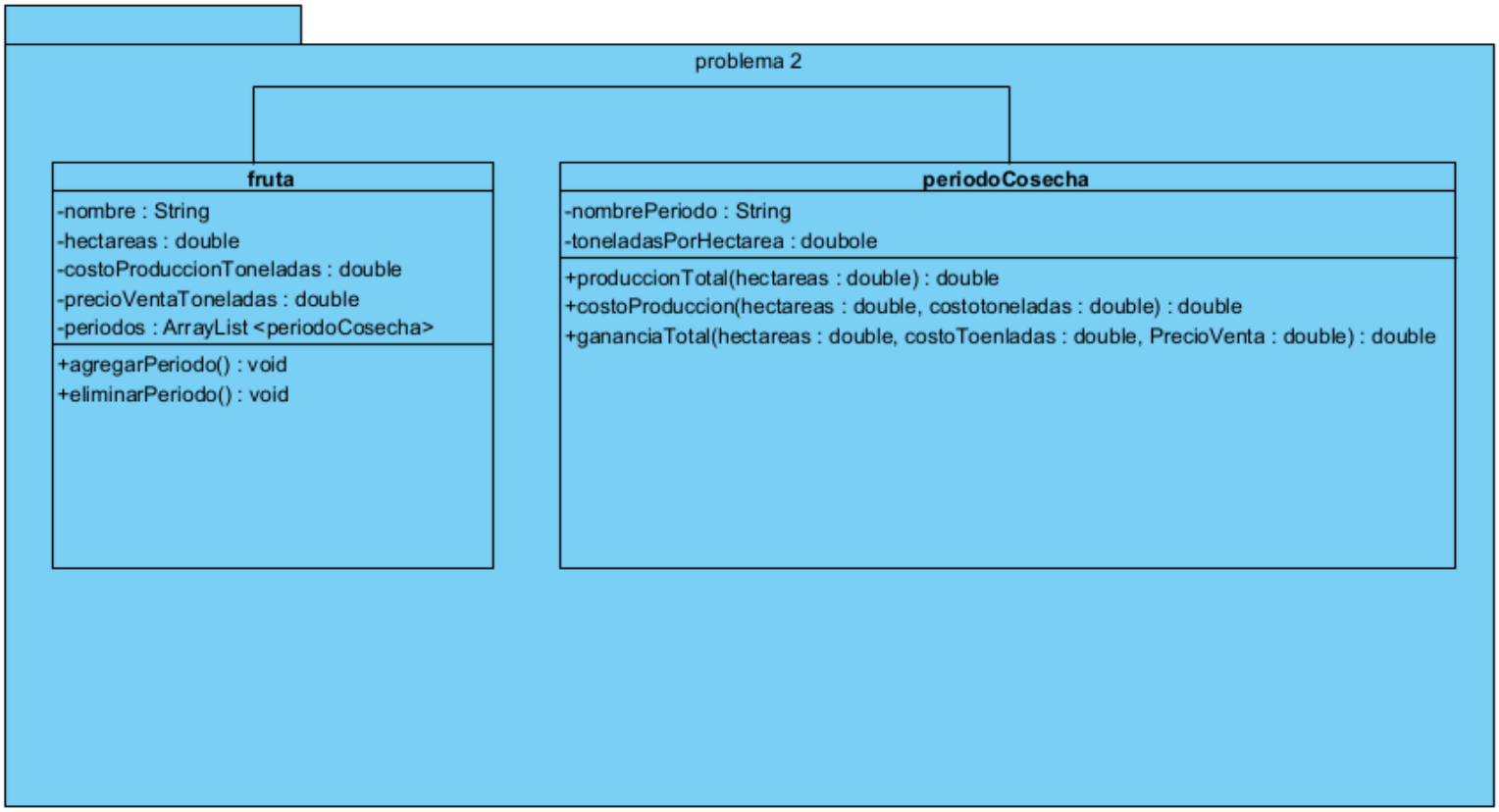
Clase Periodo Cosecha:

PeriodoCosecha			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
nombrePeriodo	String	1	Si
toneladasPorHectarea	Double	1	Si

Operaciones:

periodoCosecha			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		nombrePeriodo, toneladasPorHectarea	
produccionTotal	Double	hectáreas	Publico
costoProduccion	Double	Hectáreas, costoToneladas	Publico
gananciaTotal	Double	hectáreas, costoToneladas, precioVenta	Publico

Diagrama:



Código Clase Fruta:

```java
package empresa.agricola;

import java.util.ArrayList;
import java.util.Objects;

public class Fruta implements Comparable<Fruta> {  5 usages  Irving Emanuel Garcia Cedeño
    private String NombreFruta;  9 usages
    private double Hectareas;  5 usages
    private double CostoProduccionTon;  5 usages
    private double PrecioVentaTon;  5 usages
    private ArrayList<PeriodoCosecha> Periodos;  4 usages

    public Fruta(String nombreFruta, double hectareas, double costoProduccionTon, ArrayList<PeriodoCosecha> periodos, double precioVentaTon) {
        this.NombreFruta = nombreFruta;
        this.Hectareas = hectareas;
        this.CostoProduccionTon = costoProduccionTon;
        this.Periodos = periodos;
        this.PrecioVentaTon = precioVentaTon;
    }

    public String getNombreFruta() {  no usages  Irving Emanuel Garcia Cedeño
        return NombreFruta;
    }

    public void setNombreFruta(String nombreFruta) {  no usages  Irving Emanuel Garcia Cedeño
        NombreFruta = nombreFruta;
    }
```

Fruta.java × PeriodoCosecha.java

```java
6   public class Fruta implements Comparable<Fruta> {  5 usages  Irving Emanuel Garcia Cedeño
25      public void setNombreFruta(String nombreFruta) {  no usages  Irving Emanuel Garcia Cedeño
26          NombreFruta = nombreFruta;
27      }
28
29      public double getHectareas() {  no usages  Irving Emanuel Garcia Cedeño
30          return Hectareas;
31      }
32
33      public void setHectareas(double hectareas) {  no usages  Irving Emanuel Garcia Cedeño
34          Hectareas = hectareas;
35      }
36
37      public double getCostoProduccionTon() {  no usages  Irving Emanuel Garcia Cedeño
38          return CostoProduccionTon;
39      }
40
41      public void setCostoProduccionTon(double costoProduccionTon) {  no usages  Irving Emai
42          CostoProduccionTon = costoProduccionTon;
43      }
44
45      public double getPrecioVentaTon() {  no usages  Irving Emanuel Garcia Cedeño
46          return PrecioVentaTon;
47      }
```

Fruta.java × PeriodoCosecha.java

```java
6   public class Fruta implements Comparable<Fruta> {  5 usages  Irving Emanuel Garcia Ced
49      public void setPrecioVentaTon(double precioVentaTon) {  no usages  Irving Emanu
50          PrecioVentaTon = precioVentaTon;
51      }
52
53      public ArrayList<PeriodoCosecha> getPeriodos() {  no usages  Irving Emanuel Garci
54          return Periodos;
55      }
56
57      public void agregarPeriodo(PeriodoCosecha periodo) {  no usages  Irving Emanuel
58          Periodos.add(periodo);
59      }
60
61      public void eliminarPeriodo(PeriodoCosecha periodo) {  no usages  Irving Emanue
62          Periodos.remove(periodo);
63      }
64
65  @   public double gananciaPeriodo(PeriodoCosecha p) {  no usages  Irving Emanuel Garc
66          return p.gananciaTotal(Hectareas, CostoProduccionTon, PrecioVentaTon);
67      }
68
69      @Override  Irving Emanuel Garcia Cedeño
70  @   public int compareTo(Fruta f) {
71          return this.NombreFruta.compareTo(f.NombreFruta);
72      }
```

```java
Fruta.java ×   PeriodoCosecha.java

public class Fruta implements Comparable<Fruta> { 5 usages  Irving Emar

    @Override  Irving Emanuel Garcia Cedeño
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Fruta)) return false;
        Fruta f = (Fruta) obj;
        return NombreFruta.equals(f.NombreFruta);
    }

    @Override  Irving Emanuel Garcia Cedeño
    public int hashCode() {
        return Objects.hash(NombreFruta);
    }

    @Override  Irving Emanuel Garcia Cedeño
    public String toString() {
        return "Fruta: " + NombreFruta +
                "\nHectareas: " + Hectareas +
                "\nCosto Produccion Ton: " + CostoProduccionTon +
                "\nPrecio Venta Ton: " + PrecioVentaTon;
    }
}
```

Codigo Clase PeriodoCosecha

```java
Fruta.java   PeriodoCosecha.java ×

package empresa.agricola;

public class PeriodoCosecha { 6 usages  Irving Emanuel Garcia Cedeño
    private String NombrePeriodo; 4 usages
    private double CantidadToneladas; 5 usages

    public PeriodoCosecha(String nombrePeriodo, double cantidadToneladas) {
        this.NombrePeriodo = nombrePeriodo;
        this.CantidadToneladas = cantidadToneladas;
    }

    public String getNombrePeriodo() { no usages  Irving Emanuel Garcia Cedeño
        return NombrePeriodo;
    }

    public void setNombrePeriodo(String nombrePeriodo) { no usages  Irving Emanu
        NombrePeriodo = nombrePeriodo;
    }

    public double getCantidadToneladas() { no usages  Irving Emanuel Garcia Cedeño
        return CantidadToneladas;
    }

    public void setCantidadToneladas(double cantidadToneladas) { no usages
        CantidadToneladas = cantidadToneladas;
    }
```

```java
Fruta.java    PeriodoCosecha.java

public class PeriodoCosecha {  6 usages  Irving Emanuel Garcia Cedeño
    public double ProduccionTotal(double Hectareas) {  2 usages  Irving Emanuel Garcia Cedeño
        return CantidadToneladas*Hectareas;

    }
    public double CostoProduccion(double Hectareas, double CostoTonleadas){  no usages  Irving En
        return ProduccionTotal(Hectareas)*CostoTonleadas;
    }

    public double gananciaTotal(double Hectareas,double CostoToneladas, double precioVenta){
        double produccion=ProduccionTotal(Hectareas);
        return (produccion*precioVenta)- (produccion*CostoToneladas);
    }

    @Override  Irving Emanuel Garcia Cedeño
    public String toString() {
        return "PeriodoCosecha{" +
                "NombrePeriodo='" + NombrePeriodo + '\'' +
                ", CantidadToneladas=" + CantidadToneladas +
                '}';
    }
}
```

Prueba:

```
jshell> ArrayList<PeriodoCosecha> lista = new ArrayList<>();
lista ==> []

jshell> Fruta f1 = new Fruta("Manzana", 10, 200, lista, 300);
f1 ==> Fruta: Manzana
Hectareas: 10.0
Costo Produccion T ... .0
Precio Venta Ton: 300.0

jshell> PeriodoCosecha p1 = new PeriodoCosecha("Enero", 50);
p1 ==> PeriodoCosecha{NombrePeriodo='Enero', CantidadToneladas=50.0}

jshell> PeriodoCosecha p2 = new PeriodoCosecha("Febrero", 70);
p2 ==> PeriodoCosecha{NombrePeriodo='Febrero', CantidadToneladas=70.0}

jshell> lista.add(p1);
$20 ==> true

jshell> lista.add(p2);
$21 ==> true

jshell> Fruta f2 = new Fruta("Manzana", 10, 200, lista, 300);
f2 ==> Fruta: Manzana
Hectareas: 10.0
Costo Produccion T ... .0
Precio Venta Ton: 300.0

jshell> f1.equals(f2);
$23 ==> true
```

```
jshell> Fruta f3 = new Fruta("Naranja", 15, 180, lista, 250);
f3 ==> Fruta: Naranja
Hectareas: 15.0
Costo Produccion T ... .0
Precio Venta Ton: 250.0

jshell> f1.compareTo(f3);
$25 ==> -1

jshell> ArrayList<Fruta> frutas = new ArrayList<>();
frutas ==> []

jshell> frutas.add(f1);
$27 ==> true

jshell> frutas.add(f3);
$28 ==> true

jshell>

jshell> Collections.sort(frutas);

jshell> frutas;
frutas ==> [Fruta: Manzana
Hectareas: 10.0
Costo Produccion Ton: 200.0
Precio Venta Ton: 300.0, Fruta: Naranja
Hectareas: 15.0
Costo Produccion Ton: 180.0
Precio Venta Ton: 250.0]
```

Análisis del código:

En esta parte del programa se crea un objeto llamado fruta1, al cual se le asignan los valores correspondientes a los atributos de la clase Fruta, como el nombre de la fruta, la cantidad de hectáreas, el costo de producción por tonelada y el precio de venta por tonelada.

Posteriormente se crea un objeto de la clase PeriodoCosecha, donde se especifica el nombre del periodo y la cantidad de toneladas estimadas por hectárea. Después se utiliza el método agregarPeriodo() para agregar ese periodo de cosecha a la fruta.

Además, se crean dos objetos de tipo Fruta con los mismos valores y se comparan utilizando el método equals(), obteniendo como resultado true, lo que indica que ambos objetos son iguales porque tienen la misma información, también se incluye hashCode() para asegurar consistencia entre objetos iguales.

Posteriormente, se compara otra fruta diferente utilizando el método compareTo(), obteniendo como resultado -1, lo que indica que una fruta es menor que la otra según el criterio definido (por ejemplo, las hectáreas, costo de producción).

En la clase PeriodoCosecha se utilizan diferentes métodos para realizar los cálculos solicitados. El método produccionTotal() calcula la producción total multiplicando las toneladas por hectárea por el número de hectáreas, el método costoProduccion() calcula el costo total multiplicando la producción total por el costo de producción por tonelada, el método gananciaTotal() calcula las ganancias estimadas restando el costo total de producción al ingreso total obtenido por la venta de las toneladas producidas.

Problema 3:

El observatorio astronómico de San Pedro Mártir, de acuerdo a sus observaciones necesita un catálogo de los cuerpos celestes descubiertos para lo cual de cada cuerpo celeste se describe la ubicación en el espacio estelar en relación a las posiciones geográficas espaciales (expresado en grado de latitud (norte o sur) y grados de longitud (este o oeste)) periodo de ubicación (mes(es) donde se observó en dicha posición), distancia aproximada a la tierra (años luz o km/segundos), el nombre que se le asigno, la composición del cuerpo estelar (gases, solidos, líquidos). De cada cuerpo celeste se puede obtener el desplazamiento con respecto a otra observación del mismo cuerpo en un periodo distinto.

Análisis del problema:

Clase cuerpo celeste: La clase CuerpoCeleste representa cada cuerpo celeste registrado por el observatorio. En esta clase se guardan datos como el nombre del cuerpo celeste, su composición y la distancia aproximada a la Tierra junto con la unidad en la que se mide.

Además, esta clase incluye una operación que permite calcular el desplazamiento del cuerpo celeste comparando dos observaciones realizadas en distintos periodos.

Clase periodo visible: La clase PeriodoVisible representa el periodo en el que el cuerpo celeste fue observado en una determinada posición. En esta clase se guarda el nombre del periodo de observación y la ubicación espacial donde se encontraba el cuerpo celeste.

Clase Ubicación espacial: La clase UbicacionEspacial representa las coordenadas espaciales del cuerpo celeste. Estas coordenadas están formadas por la latitud, el hemisferio de latitud, la longitud y el hemisferio de longitud.

Esta clase incluye un método que permite calcular el desplazamiento entre dos ubicaciones espaciales diferentes.

Clases Cuerpos celestes

Cuerpos celestes			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
nombre	String	1	si
composición	String	1	si
distanciaTierra	Double	1	si
unidadTierra	String	1	si

Operaciones:

Cuerpos celestes			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		Nombre, composición, distanciaTierra, unidadTierra	
calcularDesplazamiento	Double	Dos observaciones, una primero y otra después	public

Clase periodo Visible:

Periodo Visible			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
periodo	String	1	si
ubicacion	ubicacionEspacial	1	si

Operaciones:

Periodo Visible			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		Periodo, ubicacion	
mostrarUbicacion	void		public

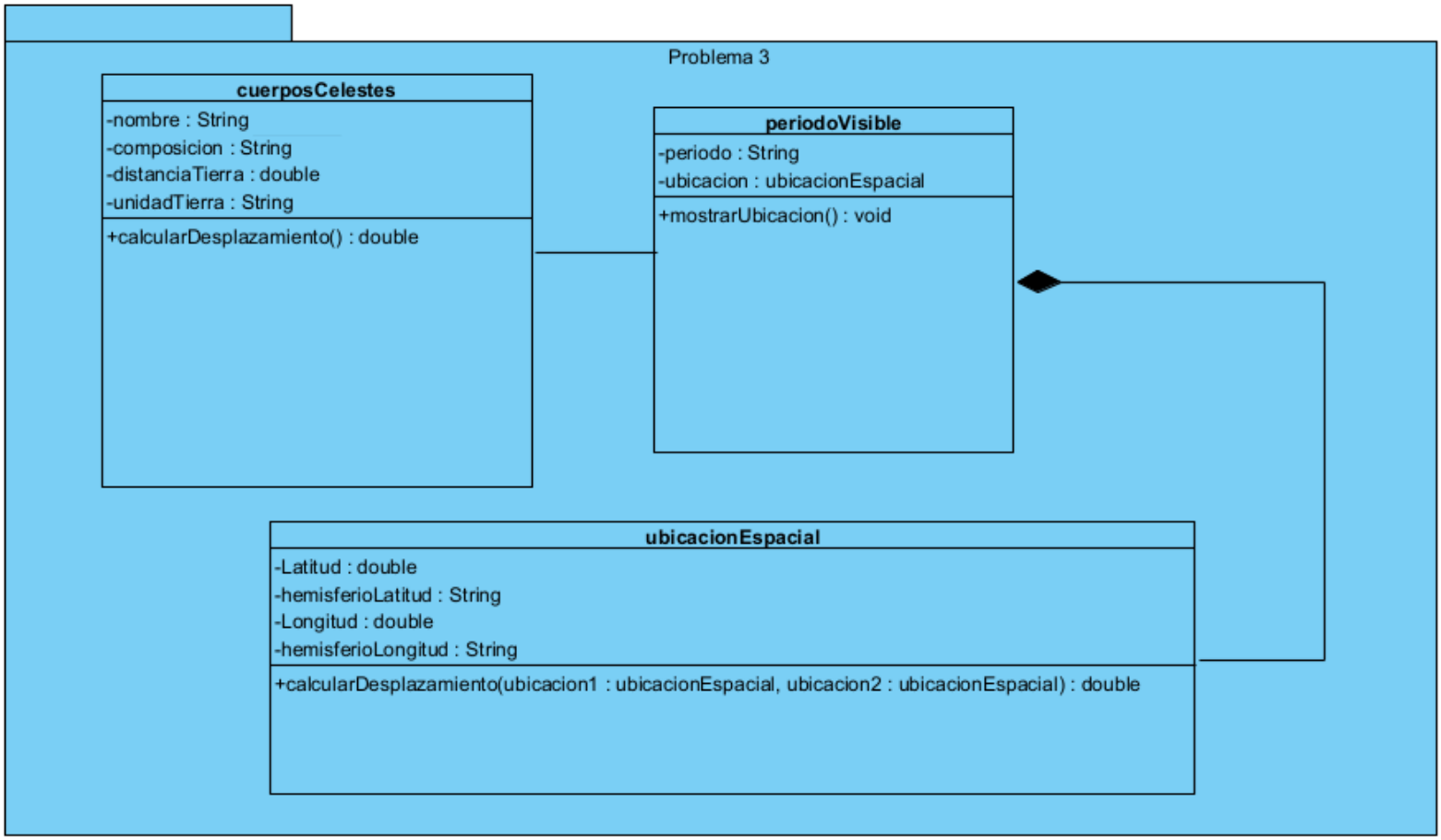
Clase Ubicación Especial

Ubicación Espacial			
Atributo	Tipo de dato	Cardinalidad	Obligatoriedad
latitud	Double	1	Si
hemisferioLatitud	String	1	Si
longitud	Double	1	Si
hemisferioLongitud	String	1	Si

Operaciones:

Ubicación Espacial			
Operación	Tipo de dato (retorno)	argumentos	Alcance
constructor		lataitud, hemisferioLatitud, longitud, hemisferioLongitud	
calcularDesplazamiento	Double	Ubicacion1, ubicacion2	Public

Diagrama:



Código Clase CuerposCelestes:

CuerposCelestes.java × PeriodoVisible.java UbicacionEspacial.java

```java
package Observatorio;

import java.util.ArrayList;
import java.util.Objects;

public class CuerposCelestes implements Comparable<CuerposCelestes> {  5 usages  Irving Emanuel Garcia Cedeño
    private String Nombre;  7 usages
    private String  Composicion;  4 usages
    private double DistanciaTierra;  4 usages
    private String UnidadTierra;  4 usages
    private ArrayList<PeriodoVisible> Periodos;  4 usages

    public CuerposCelestes(String nombre, String composicion, double distanciaTierra, String unidadTierra, ArrayList<PeriodoVisible> periodos) {
        Nombre = nombre;
        Composicion = composicion;
        DistanciaTierra = distanciaTierra;
        UnidadTierra = unidadTierra;
        Periodos = periodos;
    }

    public String getNombre() {  no usages  Irving Emanuel Garcia Cedeño
        return Nombre;
    }

    public void setNombre(String nombre) {  no usages  Irving Emanuel Garcia Cedeño
        Nombre = nombre;
    }
```

CuerposCelestes.java × PeriodoVisible.java UbicacionEspacial.java

```java
public class CuerposCelestes implements Comparable<CuerposCelestes> {
    public String getComposicion() {  no usages  Irving Emanuel Garcia Cedeño
        return Composicion;
    }

    public void setComposicion(String composicion) {  no usages  Irving En
        Composicion = composicion;
    }

    public double getDistanciaTierra() {  2 usages  Irving Emanuel Garcia Ced
        return DistanciaTierra;
    }

    public void setDistanciaTierra(double distanciaTierra) {  no usages
        DistanciaTierra = distanciaTierra;
    }

    public String getUnidadTierra() {  no usages  Irving Emanuel Garcia Cedeño
        return UnidadTierra;
    }

    public void setUnidadTierra(String unidadTierra) {  no usages  Irving
        UnidadTierra = unidadTierra;
    }
```

CuerposCelestes.java × PeriodoVisible.java UbicacionEspacial.java

```java
public class CuerposCelestes implements Comparable<CuerposCelestes> {
    public ArrayList<PeriodoVisible> getPeriodos() {  no usages  Irving Em
        return Periodos;
    }

    public void agregarPeriodo(PeriodoVisible p){  no usages  Irving Emanue
        Periodos.add(p);
    }
    public void eliminarPeriodo(PeriodoVisible p){  no usages  Irving Eman
        Periodos.remove(p);
    }
    @Override  Irving Emanuel Garcia Cedeño
    public int compareTo(CuerposCelestes c) {
        return Double.compare(this.getDistanciaTierra(), c.getDistancia
    }
    @Override  Irving Emanuel Garcia Cedeño
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof CuerposCelestes)) return false;
        CuerposCelestes c = (CuerposCelestes) obj;
        return Nombre.equals(c.Nombre);
    }
    @Override  Irving Emanuel Garcia Cedeño
    public int hashCode() {
        return Objects.hash(Nombre);
    }
```

```java
CuerposCelestes.java × PeriodoVisible.java UbicacionEspacial.java
public class CuerposCelestes implements Comparable<CuerposCelestes> {
    @Override  Irving Emanuel Garcia Cedeño
    public String toString() {
        return "cuerposCelestes{" +
                "Nombre='" + Nombre + '\'' +
                ", Composicion='" + Composicion + '\'' +
                ", DistanciaTierra=" + DistanciaTierra +
                ", UnidadTierra='" + UnidadTierra + '\'' +
                '}';
    }
}
```

Código clase ubcacionEspacial:

```java
CuerposCelestes.java PeriodoVisible.java × UbicacionEspacial.java
package Observatorio;

public class PeriodoVisible {  5 usages  Irving Emanuel Garcia Cedeño
    private String Periodo;  4 usages
    private UbicacionEspacial Ubicacion;  4 usages

    public PeriodoVisible(String periodo, UbicacionEspacial ubicacion) {
        Periodo = periodo;
        Ubicacion = ubicacion;
    }

    public String getPeriodo() {  no usages  Irving Emanuel Garcia Cedeño
        return Periodo;
    }

    public void setPeriodo(String periodo) {  no usages  Irving Emanuel Garcia C
        Periodo = periodo;
    }

    public UbicacionEspacial getUbicacion() {  no usages  Irving Emanuel Garcia
        return Ubicacion;
    }

    public void setUbicacion(UbicacionEspacial ubicacion) {  no usages  Irv
        Ubicacion = ubicacion;
    }
```

```java
public class PeriodoVisible {  5 usages  Irving Emanuel Garcia Cedeño

    @Override  Irving Emanuel Garcia Cedeño
    public String toString() {
        return "periodoVisible{" +
                "periodo='" + Periodo + '\'' +
                ", ubicacion=" + Ubicacion +
                '}';
    }
}
```

Código clase periodoVisible:

```java
package Observatorio;

public class UbicacionEspacial {  5 usages  Irving Emanuel Garcia Cedeño
    private double Latitud;  6 usages
    private String HemisferioLatitud;  3 usages
    private double Longitud;  6 usages
    private String HemisferioLongitud;  3 usages

    public UbicacionEspacial(double latitud, String hemisferioLatitud, String hemisferioLongitud, double longitud) {
        this.Latitud = latitud;
        this.HemisferioLatitud = hemisferioLatitud;
        this.HemisferioLongitud = hemisferioLongitud;
        this.Longitud = longitud;
    }

    public double getLatitud() {  no usages  Irving Emanuel Garcia Cedeño
        return Latitud;
    }

    public void setLatitud(double latitud) {  no usages  Irving Emanuel Garcia Cedeño
        Latitud = latitud;
    }

    public String getHemisferioLatitud() {  no usages  Irving Emanuel Garcia Cedeño
        return HemisferioLatitud;
    }
```

CuerposCelestes.java | PeriodoVisible.java | UbicacionEspacial.java

```java
3   public class UbicacionEspacial {  5 usages  Irving Emanuel Garcia Cedeño
28      public void setHemisferioLatitud(String hemisferioLatitud) {  no
29          HemisferioLatitud = hemisferioLatitud;
30      }
31
32      public double getLongitud() {  no usages  Irving Emanuel Garcia Cedeño
33          return Longitud;
34      }
35
36      public void setLongitud(double longitud) {  no usages  Irving Emanuel
37          Longitud = longitud;
38      }
39
40      public String getHemisferioLongitud() {  no usages  Irving Emanuel Gar
41          return HemisferioLongitud;
42      }
43
44      public void setHemisferioLongitud(String hemisferioLongitud) {
45          HemisferioLongitud = hemisferioLongitud;
46      }
47
48  @   public double calcularDesplazamiento(UbicacionEspacial otra){  no
49          double dx = this.Latitud - otra.Latitud;
50          double dy = this.Longitud - otra.Longitud;
51          return Math.sqrt(dx*dx + dy*dy);
52      }
```

CuerposCelestes.java | PeriodoVisible.java | UbicacionEspacial.java

```java
3   public class UbicacionEspacial {  5 usages  Irving Emanuel Garcia Cedeño
54      @Override  Irving Emanuel Garcia Cedeño
55      public String toString() {
56          return "ubicacionEspacial{" +
57                  "longitud=" + Longitud +
58                  ", latitud=" + Latitud +
59                  '}';
60      }
61  }
```

Prueba:

```
jshell> UbicacionEspacial u1 = new UbicacionEspacial(10.5, "Norte", "Este", 20.3);
u1 ==> ubicacionEspacial{longitud=20.3, latitud=10.5}

jshell> UbicacionEspacial u2 = new UbicacionEspacial(15.2, "Sur", "Oeste", 30.1);
u2 ==> ubicacionEspacial{longitud=30.1, latitud=15.2}

jshell> PeriodoVisible p1 = new PeriodoVisible("Enero", u1);
p1 ==> periodoVisible{periodo='Enero', ubicacion=ubicaci ... gitud=20.3, latitud=10.5}}

jshell> PeriodoVisible p2 = new PeriodoVisible("Febrero", u2);
p2 ==> periodoVisible{periodo='Febrero', ubicacion=ubica ... gitud=30.1, latitud=15.2}}

jshell> ArrayList<PeriodoVisible> lista = new ArrayList<>();
lista ==> []

jshell> lista.add(p1);
$38 ==> true

jshell> lista.add(p2);
$39 ==> true

jshell> CuerposCelestes c1 = new CuerposCelestes("Marte", "Rocoso", 300, "km", lista);
c1 ==> cuerposCelestes{Nombre='Marte', Composicion='Roco ... =300.0, UnidadTierra='km'}

jshell> CuerposCelestes c2 = new CuerposCelestes("Venus", "Gaseoso", 200, "km", lista);
c2 ==> cuerposCelestes{Nombre='Venus', Composicion='Gase ... =200.0, UnidadTierra='km'}

jshell> c1.compareTo(c2);
$42 ==> 1
```

```
jshell> c1.compareTo(c2);
$42 ==> 1

jshell> c1.equals(c2);
$43 ==> false

jshell> ArrayList<CuerposCelestes> listaC = new ArrayList<>();
listaC ==> []

jshell> listaC.add(c1);
$45 ==> true

jshell> listaC.add(c2);
$46 ==> true

jshell>

jshell> Collections.sort(listaC);

jshell> listaC;
listaC ==> [cuerposCelestes{Nombre='Venus', Composicion='Gaseoso', DistanciaTierra=200.0, UnidadTierra='km'}, cuerposCelestes{Nombre='Marte', Composicion='Roc
oso', DistanciaTierra=300.0, UnidadTierra='km'}]

jshell> c1.agregarPeriodo(new PeriodoVisible("Marzo", u1));

jshell> c1.getPeriodos();
$50 ==> [periodoVisible{periodo='Enero', ubicacion=ubicacionEspacial{longitud=20.3, latitud=10.5}}, periodoVisible{periodo='Febrero', ubicacion=ubicacionEspac
ial{longitud=30.1, latitud=15.2}}, periodoVisible{periodo='Marzo', ubicacion=ubicacionEspacial{longitud=20.3, latitud=10.5}}]
```

Análisis del código:

En esta parte del programa primero se crea un objeto de la clase CuerposCelestes, donde se registran datos como el nombre del cuerpo celeste, su composición, la distancia aproximada a la Tierra y la unidad de medida.

Después se crea un objeto de la clase UbicacionEspacial, donde se especifican las coordenadas de latitud y longitud junto con sus respectivos hemisferios. Posteriormente se utilizan los métodos de la clase para calcular el desplazamiento entre dos ubicaciones diferentes del mismo cuerpo celeste.

Además, se crean dos objetos de tipo CuerposCelestes para compararlos utilizando el método equals(). Si sus atributos son iguales, el resultado es true; en caso contrario, es false, lo que indica si representan el mismo cuerpo celeste. También se implementa hashCode() para mantener consistencia en la comparación de objetos.

Posteriormente, se utiliza el método compareTo() para comparar dos cuerpos celestes, obteniendo valores 1, lo que indica cuál es mayor o menor (por ejemplo, la distancia a la Tierra).

Finalmente se imprimen los datos de los objetos creados para verificar que la información se haya almacenado correctamente y que tanto los cálculos como los métodos de comparación funcionan adecuadamente.

CONCLUSION:

En esta actividad se entendió mejor cómo aplicar la programación orientada a objetos, sobre todo al momento de definir bien las clases y sus atributos para que todo funcione correctamente. También se vio la importancia de métodos como equals() y hashCode(), ya que permiten comparar objetos de forma lógica y saber cuándo realmente son iguales o no. Además, al implementar Comparable con compareTo(), se aprendió cómo ordenar objetos según algún criterio, como el costo, nombre o distancia, lo cual facilita trabajar con listas usando métodos como Collections.sort(), en general, la actividad ayudó a comprender mejor cómo estructurar y comparar objetos, y cómo estos detalles hacen que el programa sea más completo y funcional.

URl GIT:

https://github.com/irvingemanuel2003-a11y/actividad5.git

https://github.com/soysantiagorb/ACTIVIDAD5.git